RAG-BASED CUSTOMER SUPPORT ASSISTANT

Technical Documentation

Prepared by: Rahul Pravin Dhumal

Intern ID: IN226095102

Document Type	Technical Documentation
Project Title	RAG-Based Customer Support Assistant
Architecture Layers	HLD + LLD
Workflow Engine	LangGraph (3-Node Graph)
API Framework	FastAPI — POST /ask
LLM Provider	Groq (llama-3.3-70b-versatile)
Vector Store	ChromaDB
Embedding Model	HuggingFace all-MiniLM-L6-v2

1. Introduction

1.1 What is RAG (Retrieval-Augmented Generation)?

Retrieval-Augmented Generation (RAG) is an advanced AI architecture that combines the power of large language models (LLMs) with a domain-specific knowledge base. Unlike standalone LLMs that rely solely on parametric knowledge baked in during training, RAG systems dynamically retrieve relevant information from an external document store at query time and inject it as context into the LLM prompt before generating a response.

The RAG pipeline can be summarized in three stages:

- Indexing: Documents are chunked, embedded, and stored in a vector database.
- Retrieval: At query time, the user's query is embedded and the top-k semantically similar chunks are fetched.
- Generation: The retrieved chunks are appended to the prompt, providing grounded, factual context for the LLM to generate a precise answer.

1.2 Why RAG is Needed

Traditional LLMs, while powerful, suffer from critical limitations in production customer support systems:

- Hallucination: LLMs can generate confident but factually incorrect answers when they lack domain-specific knowledge.
- Stale Knowledge: LLMs have a training cutoff and cannot access real-time or organization-specific data without fine-tuning.
- Cost of Fine-Tuning: Continually retraining or fine-tuning LLMs on new knowledge is prohibitively expensive.
- Lack of Explainability: Without RAG, it is difficult to trace why the model gave a specific answer.

RAG addresses all of these challenges by externalizing the knowledge store — making the system's knowledge base easily updatable, traceable, and cost-effective without retraining the model.

1.3 Use Case Overview

This system is designed for an e-commerce customer support scenario where users submit queries about orders, refunds, accounts, and payment issues. The assistant consults a PDF knowledge base, retrieves the most relevant support documentation, and generates a contextual answer. When the system lacks confidence, it gracefully escalates to a human agent via the Human-in-the-Loop (HITL) mechanism.

Key capabilities of the system:

- Automated resolution of high-frequency, low-complexity support queries
- Semantic similarity search over a curated PDF knowledge base
- Confidence-based routing to LLM or human escalation
- Stateful multi-turn conversations using memory
- REST API interface for integration with any frontend

2. System Architecture Explanation

2.1 High-Level Architecture Overview

The system is composed of two major pipelines that together form a complete RAG solution:

Document Ingestion Pipeline (Offline)

This is the pre-processing pipeline that runs offline before the system handles queries. It transforms raw PDF knowledge base content into a searchable vector store.

Stage	Component	Output
Step 1	PyPDFLoader reads PDF	Raw Document objects with page content + metadata
Step 2	RecursiveCharacterTextSplitter	500-char chunks with 50-char overlap
Step 3	HuggingFace all-MiniLM-L6-v2	384-dimensional float vectors per chunk
Step 4	ChromaDB Vector Store	Persisted embeddings ready for similarity search

Query Processing Pipeline (Online)

This is the real-time pipeline that activates when a user submits a query via the FastAPI endpoint. It orchestrates retrieval, routing, LLM generation, and HITL escalation through the LangGraph workflow engine.

Stage	Component	Action
Step 1	FastAPI POST /ask	Receives and validates the user query
Step 2	LangGraph Engine	Invokes the process_node with initial state
Step 3	ChromaDB Retriever	Returns top-k semantically similar chunks
Step 4	Routing Layer	Evaluates intent + score → routes to answer or HITL
Step 5a	Groq LLM Node	Generates grounded answer using context + memory
Step 5b	HITL Node	Logs to human_review_queue.txt, returns escalation ACK
Step 6	FastAPI Response	Returns JSON with query + answer fields

2.2 Component Interactions

The following table summarizes the key components, the technologies used, and how they interact with other parts of the system:

Component	Technology	Interacts With	Purpose
User Interface	FastAPI + Swagger	Graph Engine, API Layer	Entry point for all user queries
Document Loader	PyPDFLoader (LangChain)	Chunker	Reads and parses PDF knowledge base
Text Chunker	RecursiveCharacterTextSplitter	Loader, Embedder	Splits text into overlapping chunks
Embedding Model	HuggingFace all-MiniLM-L6-v2	Chunker, ChromaDB	Converts text to 384-dim vectors
Vector Store	ChromaDB	Embedder, Retriever	Stores and indexes embeddings
Retriever	ChromaDB Retriever + Reranker	Vector Store, process_node	Fetches + reranks relevant chunks
LLM	Groq llama3-8b-8192	answer_node, Memory	Generates contextual answers
Workflow Engine	LangGraph	All Nodes	Orchestrates stateful execution
Routing Layer	Custom Python Logic	process_node, answer_node, hitl_node	Routes based on score + intent
HITL Module	File-based Queue	hitl_node	Logs escalated queries for human review
Memory Module	Custom in-memory conversation buffer	answer_node, process_node	Stores conversation turns

3. Design Decisions

3.1 Chunk Size Choice

Chunk size is one of the most critical hyperparameters in a RAG system. It directly impacts retrieval precision, context quality, and LLM prompt length.

Design choice: 500-character chunks with 50-character overlap.

Parameter	Value	Rationale
chunk_size	500 characters	Balances semantic coherence with retrieval granularity. Large enough to preserve context, small enough for precise matching.
chunk_overlap	50 characters	Prevents loss of context at chunk boundaries, especially for multi-sentence facts that span boundaries.
Splitter	RecursiveCharacterTextSplitter	Splits at natural sentence/paragraph boundaries (\n\n, \n, spaces) before falling back to character splitting.

Smaller chunks (e.g., 200 characters) improve precision but lose surrounding context. Larger chunks (e.g., 1500 characters) retain more context but reduce retrieval specificity and may exceed LLM context limits. The 500-character choice represents a practical optimum for short-form customer support FAQs.

3.2 Embedding Strategy

The system uses HuggingFace's all-MiniLM-L6-v2 model for generating sentence embeddings.

- Model: all-MiniLM-L6-v2 (HuggingFace Sentence Transformers)
- Output dimension: 384-dimensional dense vectors
- Similarity metric: Cosine similarity in ChromaDB
- Deployment: Fully offline — no external API calls, no cost per embedding

This model was chosen over paid alternatives (e.g., OpenAI Embeddings at $0.0001/1K tokens) because it runs locally, provides strong semantic similarity performance for English text, and requires no authentication. For a production system with multilingual support, a model such as paraphrase-multilingual-MiniLM-L12-v2 would be considered.

3.3 Retrieval Approach

The retrieval strategy uses cosine similarity search over ChromaDB, followed by reranking:

- Top-k retrieval: The top-k most similar document chunks are fetched from ChromaDB based on cosine similarity between the query embedding and stored chunk embeddings.
- Reranking: Retrieved chunks are scored and sorted by relevance score. The average score of the top-k chunks forms the confidence score used in routing.
- Threshold: A threshold of 1.8 is used based on ChromaDB L2 distance.Higher score indicates lower similarity.

Approximate Nearest Neighbor (ANN) search in ChromaDB returns results in under 50ms, making it suitable for real-time query processing without a dedicated search infrastructure.

3.4 Prompt Design Logic

The LLM prompt is carefully structured to maximize accuracy and minimize hallucination:

```
You are a professional customer support assistant.
Rules:
- Answer ONLY using the provided context
- Be clear and concise
- If unsure, say you are not confident

Previous Conversation: {memory}

Context: {retrieved_chunks}

Question: {user_query}

Answer:
```

The key design decisions in the prompt are:

- Strict grounding: The model is instructed to answer only from the provided context, preventing hallucination.
- Memory injection: Prior conversation turns are injected before the context, enabling multi-turn coherence.
- Uncertainty acknowledgement: The model is explicitly instructed to express low confidence rather than guess.

4. Workflow Explanation

4.1 LangGraph Usage

LangGraph is a library built on top of LangChain that enables the construction of stateful, multi-actor workflows as directed graphs. It was selected for this project because it provides:

- Stateful graph execution with a shared state dictionary passed between nodes
- Conditional edge routing based on state values (e.g., the 'route' field)
- Clean separation of concerns — each node is a self-contained Python function
- Built-in support for cycles and human-in-the-loop interruption points

The workflow is defined as a StateGraph with three nodes and conditional edges that determine execution flow based on the route field in the graph state.

4.2 Node Responsibilities

Node	Responsibility	Input	Output
process_node	Retrieves context, calculates similarity score, detects intent, and decides routing.	User query string	GraphState with context, score, intent, and route
answer_node	Constructs the LLM prompt with memory and context, invokes Groq API, saves response to memory.	GraphState with context + route='answer'	GraphState with final response
hitl_node	Formats and appends an escalation log entry to human_review_queue.txt, returns escalation acknowledgment.	GraphState with route='hitl'	GraphState with escalation message as response

4.3 State Transitions

The graph state is a typed Python dictionary (TypedDict) that flows through all nodes:

```
{
  "query":    str,   # Original user query
  "context":  str,   # Retrieved document chunks
  "route":    str,   # "answer" or "hitl"
  "response": str    # Final response to return
}
```

State transition flow:

```
START
  → process_node  (fills: context, route)
  → [conditional edge based on state['route']]
      'answer' → answer_node  (fills: response)
      'hitl'   → hitl_node    (fills: response)
  → END
```

5. Conditional Logic

5.1 Intent Detection

Intent detection is performed using keyword matching within the process_node function. The query string is converted to lowercase and scanned against predefined keyword sets for each support category:

Intent Category	Trigger Keywords	Routing Bias
order	order, tracking, delivery, shipment, shipped, arrive	Answer (if score sufficient)
refund	refund, return, money back, cancel, reimburse	Answer (if score sufficient)
account	account, login, password, profile, email	Answer (if score sufficient)
payment	payment, billing, charge, invoice, card, deducted	Answer (if score sufficient)
unknown	Any query not matching the above categories	HITL (escalate immediately)

Intent detection is deliberately kept simple (keyword matching) for speed and interpretability. A production system could replace this with a lightweight text classification model (e.g., a fine-tuned BERT classifier) for higher accuracy, but keyword matching is sufficient given the well-defined and limited support vocabulary.

5.2 Routing Decisions

The routing decision in process_node follows a multi-factor logic combining intent, similarity score, and memory presence:

Condition	Route	Reason
intent == 'unknown' AND memory is empty	HITL	Query is off-topic and no conversation context exists to help.
similarity score > 1.8 AND memory is empty	HITL	Score above 1.8 indicates poor retrieval (ChromaDB L2 distance, not cosine). Low confidence with no memory fallback.
score <= 1.8 OR memory has prior turns	Answer	Retrieval is confident, or prior conversation context can supplement.

Note on score interpretation: ChromaDB's default distance metric is L2 (Euclidean) distance. Lower scores indicate greater similarity. A score above 1.8 means the retrieved chunks are not semantically close to the query, which is treated as low confidence. The threshold of 1.8 was empirically determined for the all-MiniLM-L6-v2 embedding space.

6. HITL (Human-in-the-Loop) Implementation

6.1 Role of Human Intervention

The HITL module serves as the system's graceful degradation mechanism. Rather than generating a potentially incorrect or hallucinated answer when confidence is low, the system routes the query to a human agent for review and manual response.

HITL is triggered under three conditions:

- Unknown Intent: The query cannot be classified into any known support category, suggesting it falls outside the system's knowledge domain.
- Low Confidence Retrieval: The average similarity score of the top-k retrieved chunks falls below the SIMILARITY_THRESHOLD, meaning the knowledge base does not contain sufficiently relevant information.
- Missing Context: No relevant chunks are retrieved from ChromaDB, resulting in an empty or too-short context string.

6.2 HITL Flow

When the hitl_node is activated, the following sequence executes:

```
1. process_node sets state['route'] = 'hitl'
2. LangGraph conditional edge routes to hitl_node
3. hitl_node formats a log entry:
     - Timestamp (datetime.now())
     - Original query
     - Status: 'Pending Human Review'
4. Log entry is appended to human_review_queue.txt
5. Escalation ACK returned to user via FastAPI
```

The user-facing response is: "Your query has been escalated to our support team. Escalation Triggered.

Ticket added to Human Review Queue.

6.3 Benefits and Limitations

Aspect	Benefits	Limitations
Accuracy	Prevents hallucinated answers on unknown or complex queries	Increases human workload for edge-case queries
User Trust	Sets clear expectations via escalation acknowledgment	24-hour SLA may not meet real-time support needs
Implementation	Simple file-based queue — zero infrastructure overhead	Not scalable; a ticket system (Zendesk, Jira) is needed for production
Observability	Every HITL entry is logged with timestamp, enabling analysis	No automated feedback loop to improve the system from HITL data
Coverage	Gracefully handles novel query types outside the KB	Cannot distinguish between truly novel queries and retrieval failures

7. Challenges and Trade-offs

7.1 Retrieval Accuracy vs. Speed

ChromaDB uses Approximate Nearest Neighbor (ANN) search rather than exact nearest neighbor search. ANN trades a small amount of retrieval accuracy for dramatically faster query times.

- Exact KNN search would guarantee finding the globally closest vectors but scales poorly (O(n) per query).
- ANN search (HNSW index in ChromaDB) returns results in under 50ms even for large collections, with a typical recall rate above 95%.
- For customer support with a relatively small knowledge base (< 100,000 chunks), the accuracy difference between exact and approximate search is negligible.

7.2 Chunk Size vs. Context Quality

The 500-character chunk size was chosen as a practical balance, but it involves inherent trade-offs:

Chunk Size	Retrieval Precision	Context Quality	Prompt Length
Small (200 chars)	High — precise matches	Low — may lose surrounding context	Short — more chunks fit
Medium (500 chars)	Balanced — our choice	Good — preserves local context	Moderate
Large (1500 chars)	Low — over-broad matches	High — rich context per chunk	Long — few chunks, may exceed limits

The 50-character overlap mitigates the boundary context loss of the 500-character chunks, reducing cases where a relevant sentence is split across two chunks and neither fully retrieved.

7.3 Cost vs. Performance

The system was designed to minimize operational costs while maintaining strong performance:

Component	Chosen Solution	Cost	Alternative
Embeddings	HuggingFace all-MiniLM-L6-v2	Free (local)	OpenAI Embeddings ($0.0001/1K tokens)
LLM Inference	Groq (llama3-8b-8192)	Free tier available, < 1s latency	OpenAI GPT-4 ($10/1M tokens)
Vector DB	ChromaDB (local)	Free, no managed service	Pinecone ($70+/month)
API Framework	FastAPI (self-hosted)	Free	AWS API Gateway ($3.50/1M requests)

The total infrastructure cost for local development is zero, making this architecture highly accessible for prototyping and small-scale deployment. The primary performance trade-off is that Groq's free tier has rate limits; under high load, response times could degrade. A production deployment would use a paid Groq tier or self-host Llama via Ollama.

8. Testing Strategy

8.1 Testing Approach

The testing strategy covers each module independently (unit testing) and the end-to-end query flow (integration testing).

Unit Testing

- **loader.py**: Verify load_and_chunk() returns a non-empty list of Document objects with populated page_content and metadata fields.
- **embedder.py**: Assert that embeddings are 384-dimensional float vectors and that the ChromaDB collection has a non-zero document count after ingestion.
- **retriever.py**: Submit a known query and assert that the top-1 retrieved chunk contains semantically related content; assert similarity score is within expected range.
- **graph.py**: Test process_node with a known-good query (should route to 'answer') and an unknown query with empty memory (should route to 'hitl').
- **hitl.py**: Assert that escalate_to_human() appends a correctly formatted entry to human_review_queue.txt.
- **memory.py**: Assert that save_to_memory() correctly stores a turn and that get_memory_context() retrieves it in the next call.

Integration Testing

- Submit queries via the FastAPI POST /ask endpoint using pytest + httpx.
- Assert HTTP 200 response for all valid queries.
- Assert JSON response contains 'query' and 'answer' fields.
- Assert HITL queries result in the escalation acknowledgment string in the response.

8.2 Sample Test Queries

Query	Expected Route	Expected Behavior
What is your refund policy for damaged items?	Answer (LLM)	Retrieves refund chunks; generates policy answer from context
How do I track my order #12345?	Answer (LLM)	Retrieves order tracking chunks; generates step-by-step tracking instructions
I forgot my account password, how do I reset it?	Answer (LLM)	Retrieves account/password chunks; explains password reset procedure
My payment was deducted but no order was placed.	Answer (LLM)	Retrieves payment chunks; explains dispute resolution steps
How does your enterprise SLA agreement work?	HITL	Intent = unknown, no memory; routes to HITL; logs to queue
[Empty string]	Validation Error	FastAPI Pydantic validation returns HTTP 422 before graph invocation
What is the meaning of life?	HITL	Intent = unknown; routes to HITL escalation

8.3 Regression Testing

For each code change, the following core scenarios should be re-validated:

- Ingestion pipeline produces a non-empty ChromaDB collection.
- At least one known refund query routes to the LLM node.
- At least one unknown query routes to the HITL node.
- Memory correctly persists across at least two consecutive API calls in the same session.

9. Future Enhancements

9.1 Multi-Document Support

The current system supports a single PDF knowledge base. A production deployment would require multi-document ingestion with the following enhancements:

- Support ingestion of multiple PDFs, HTML pages, Word documents, and database records.
- Implement document versioning — track when a chunk was ingested and from which source document.
- Add metadata filtering to the retrieval step (e.g., retrieve only from the 'returns-policy' document for refund queries).
- Introduce incremental re-indexing to update the vector store when knowledge base documents change, without requiring full re-ingestion.

9.2 Feedback Loop

A feedback loop would close the gap between system performance and ground truth, enabling continuous improvement:

- Collect implicit feedback via user signals (e.g., follow-up queries indicating dissatisfaction).
- Build a labeled dataset from HITL resolutions — each human-resolved query becomes a training example.
- Use this dataset to fine-tune the embedding model or the intent classifier for improved domain performance.
- Implement an A/B testing framework to compare routing threshold configurations.

9.3 Enhanced Memory Integration

The current ConversationBufferMemory stores all prior turns, which can grow unboundedly and eventually exceed the LLM's context window:

- Replace ConversationBufferMemory with ConversationSummaryMemory — the LLM periodically summarizes older conversation turns into a compressed summary.
- Add user session management so that memory is scoped per user session and persisted in a database (Redis or PostgreSQL).
- Implement memory-aware routing — prior turns about a specific topic should lower the HITL escalation threshold for related follow-up queries.

9.4 Production Deployment

The current architecture is designed for local development. A production deployment would require:

Concern	Current State	Production Solution
Containerization	Local Python execution	Docker + docker-compose (multi-service setup)
Backend Hosting	Localhost (uvicorn)	Render / Railway / AWS EC2 (FastAPI deployment)
Frontend	Not implemented (API only)	React UI hosted on Vercel / Netlify
Vector DB	Local ChromaDB	ChromaDB (persisted) OR Pinecone (managed)
LLM Inference	Groq free tier	Groq paid tier OR self-hosted Llama via Ollama / vLLM
HITL Queue	Flat file (human_review_queue.txt)	Database-backed queue OR Zendesk / Jira integration
Authentication	None (dev mode)	JWT / OAuth2 via FastAPI middleware
Monitoring	Console logs	Prometheus + Grafana OR centralized logging system
Scalability	Single-process FastAPI	Dockerized services + horizontal scaling (Kubernetes optional)
CI/CD	Manual deployment	GitHub Actions for automated build & deployment
Version Control	Local Git	GitHub repository with versioning and documentation

The proposed deployment architecture ensures scalability, maintainability, and ease of integration with modern cloud platforms. The system can evolve from a local prototype to a production-ready service with minimal architectural changes.

9.5 Real-Time Chat UI

The current system exposes only a REST API with Swagger UI for testing. To enhance usability and provide a production-ready experience, a frontend interface will be developed.

Planned UI Features:

1. React-based chat interface with a clean messaging layout
2. Input box for user queries and real-time response display
3. Integration with FastAPI backend via REST API (/ask endpoint)
4. Message history display for conversation tracking
5. Loading indicator for better user experience

Future Enhancements:

- WebSocket integration for streaming responses
- Admin dashboard for HITL monitoring
- User authentication and session-based memory

This system demonstrates a complete end-to-end RAG pipeline with workflow-based routing, human-in-the-loop escalation, and memory-aware query handling, making it suitable for real-world customer support automation.